

Tugas Praktikum 9
Sistem Informasi Geografis

Nama : Awi Septian Prasetyo
NIM : 123140201
Kelas : Sistem Informasi Geografis
Dosen : Muhammad Habib Alghifari, S.kom., M.T.I.
Alya Khairunnisa Rizkita, S.Kom., M.Kom.
Github : https://github.com/Awesome1209/sig_123140201.git

Deskripsi Tugas

Kembangkan aplikasi WebGIS full-stack dengan fitur autentikasi dan CRUD untuk mengelola data fasilitas publik di wilayah Anda.

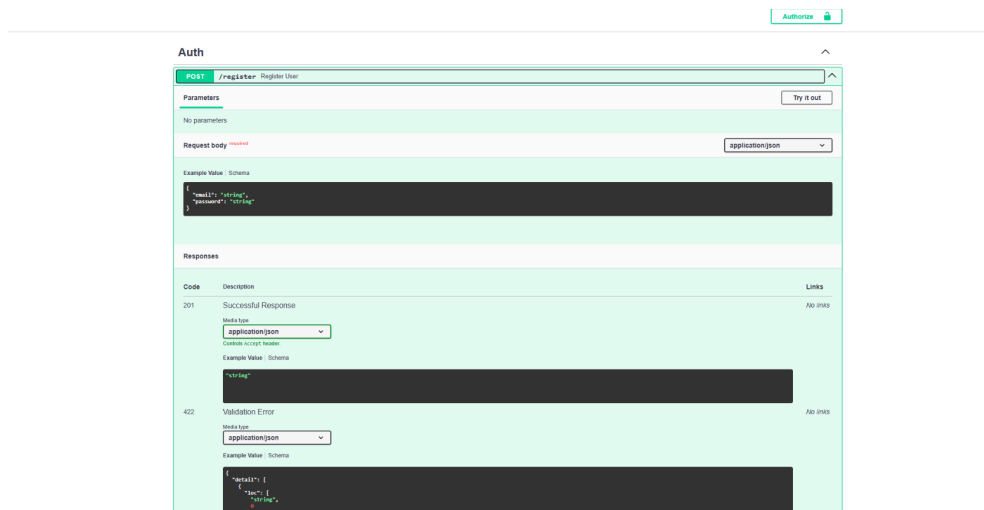
Ketentuan

- Backend: Implementasi CRUD lengkap dengan validasi Pydantic
- Auth: Implementasi login dengan JWT (minimal register & login)
- Frontend: Form untuk add/edit fasilitas dengan validasi
- Peta: Tampilkan marker yang bisa diklik untuk edit/delete
- Dokumentasi: README dengan cara setup dan screenshot

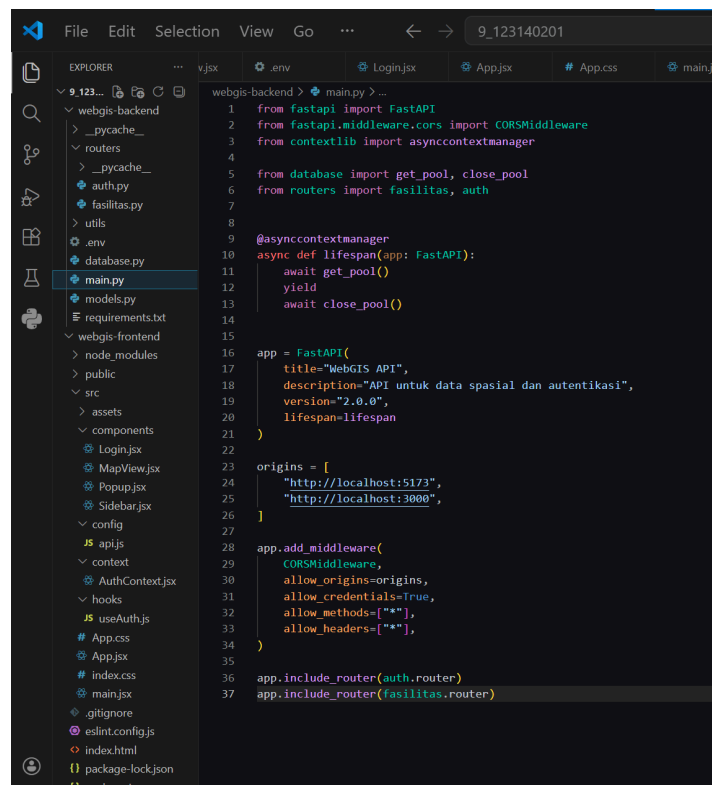
Langkah-langkah Pengerjaan:

1. Menyiapkan Backend FastAPI

Backend FastAPI yang sudah dibuat pada tugas sebelumnya digunakan kembali sebagai dasar. File `main.py` berfungsi sebagai entry point aplikasi, `database.py` sebagai penghubung ke PostgreSQL/PostGIS, `models.py` sebagai schema Pydantic, dan `routers/fasilitas.py` sebagai endpoint utama data fasilitas. Pada tugas ini kemudian ditambahkan `routers/auth.py` dan `utils/auth.py` untuk mendukung autentikasi JWT. `main.py` juga memuat lifecycle koneksi database dan registrasi router.



Gambar File utama backend FastAPI

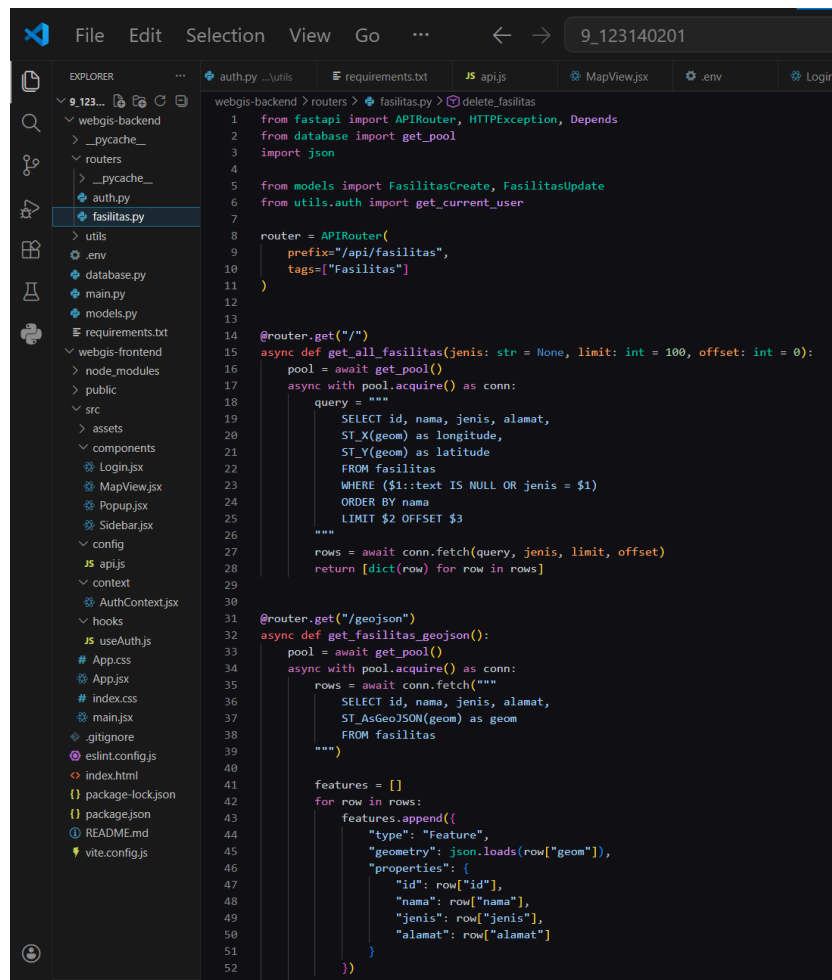


2. Menambahkan CRUD Lengkap pada Backend

Pada tugas sebelumnya backend sudah memiliki operasi dasar untuk membaca data dan menambahkan data fasilitas. Pada Praktikum 9, backend dilengkapi menjadi CRUD penuh, yaitu:

- POST /api/fasilitas/ untuk create,
- GET /api/fasilitas/ untuk list data,
- GET /api/fasilitas/geojson untuk data peta,
- GET /api/fasilitas/{id} untuk detail data,
- PUT /api/fasilitas/{id} untuk update,
- DELETE /api/fasilitas/{id} untuk delete.

Materi Pertemuan 9 memang mewajibkan implementasi CRUD lengkap dengan validasi Pydantic, dan mencontohkan operasi create, read, update, dan delete sebagai bagian inti tugas.



```
File Edit Selection View Go ... 9_123140201
EXPLORER webgis-backend > routers > fasilitas.py > delete_fasilitas
1 from fastapi import APIRouter, HTTPException, Depends
2 from database import get_pool
3 import json
4
5 from models import FasilitasCreate, FasilitasUpdate
6 from utils.auth import get_current_user
7
8 router = APIRouter(
9     prefix="/api/fasilitas",
10    tags=["Fasilitas"]
11)
12
13
14 @router.get("/")
15 async def get_all_fasilitas(jenis: str = None, limit: int = 100, offset: int = 0):
16     pool = await get_pool()
17     async with pool.acquire() as conn:
18         query = """
19             SELECT id, nama, jenis, alamat,
20                ST_X(geom) as longitude,
21                ST_Y(geom) as latitude
22             FROM fasilitas
23             WHERE ($1::text IS NULL OR jenis = $1)
24             ORDER BY nama
25             LIMIT $2 OFFSET $3
26         """
27         rows = await conn.fetch(query, jenis, limit, offset)
28         return [dict(row) for row in rows]
29
30
31 @router.get("/geojson")
32 async def get_fasilitas_geojson():
33     pool = await get_pool()
34     async with pool.acquire() as conn:
35         rows = await conn.fetch("""
36             SELECT id, nama, jenis, alamat,
37                ST_AsGeoJSON(geom) as geom
38             FROM fasilitas
39         """)
40
41     features = []
42     for row in rows:
43         features.append({
44             "type": "Feature",
45             "geometry": json.loads(row["geom"]),
46             "properties": {
47                 "id": row["id"],
48                 "nama": row["nama"],
49                 "jenis": row["jenis"],
50                 "alamat": row["alamat"]
51             }
52         })
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
```

Gambar Endpoint CRUD fasilitas pada backend

WebGIS API 2.0.0 OAS 3.1

/openapi.json

API untuk data spasial dan autentikasi

Authorize 

Auth

POST	/register	Register User	⌵
POST	/login	Login	⌵
GET	/me	Read Current User	⌵ 

Schemas

Body_login_login_post > Expand all object

HTTPValidationError > Expand all object

UserRegister > Expand all object

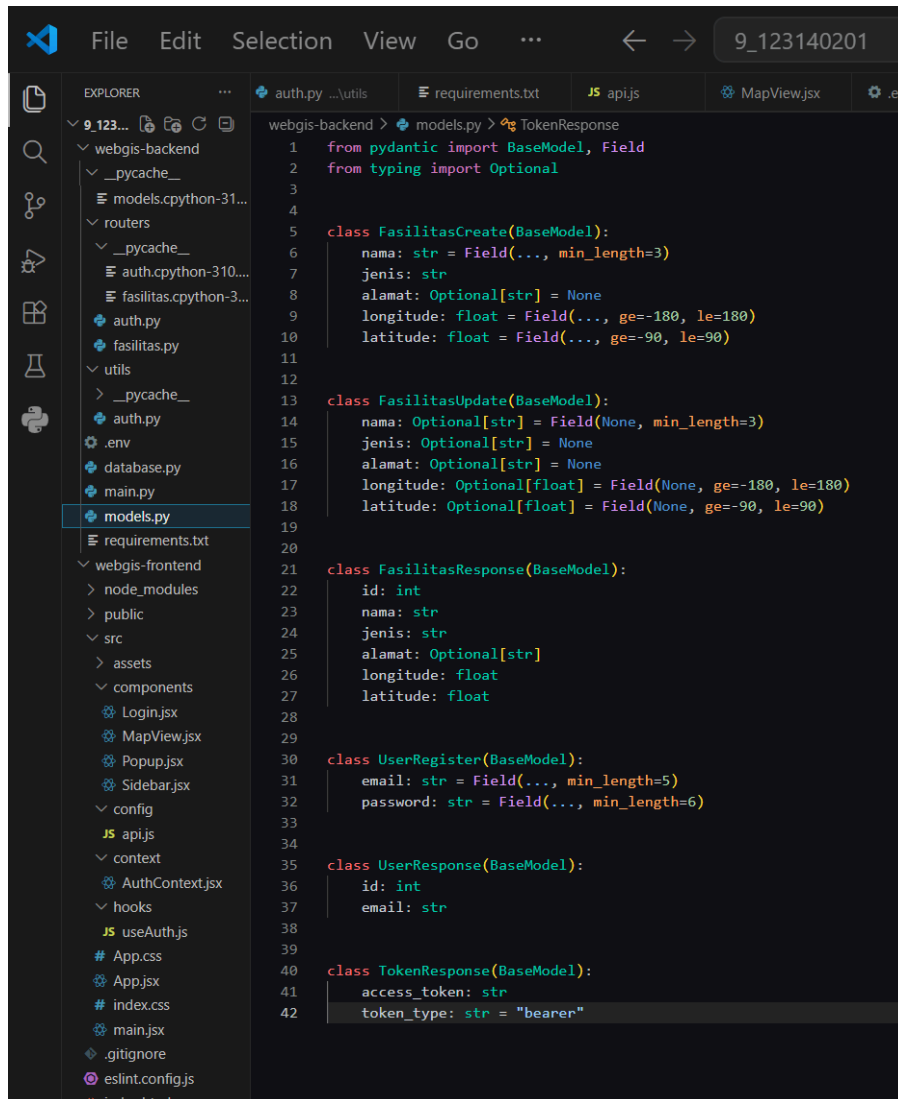
ValidationError > Expand all object

3. Menambahkan Model Validasi Pydantic

Agar input data lebih terkontrol, file models.py dikembangkan dengan beberapa schema, yaitu:

- FasilitasCreate
- FasilitasUpdate
- FasilitasResponse
- UserRegister
- TokenResponse

Model FasilitasCreate digunakan untuk tambah data, sedangkan FasilitasUpdate digunakan untuk edit data dengan field yang bersifat opsional. Ini sesuai dengan konsep validasi Pydantic yang menjadi bagian ketentuan tugas. Model awal yang sudah ada pada project sebelumnya memang memuat validasi untuk nama, longitude, dan latitude, lalu pada tugas ini ditambahkan model update dan auth.



```
1 from pydantic import BaseModel, Field
2 from typing import Optional
3
4
5 class FasilitasCreate(BaseModel):
6     nama: str = Field(..., min_length=3)
7     jenis: str
8     alamat: Optional[str] = None
9     longitude: float = Field(..., ge=-180, le=180)
10    latitude: float = Field(..., ge=-90, le=90)
11
12
13 class FasilitasUpdate(BaseModel):
14     nama: Optional[str] = Field(None, min_length=3)
15     jenis: Optional[str] = None
16     alamat: Optional[str] = None
17     longitude: Optional[float] = Field(None, ge=-180, le=180)
18     latitude: Optional[float] = Field(None, ge=-90, le=90)
19
20
21 class FasilitasResponse(BaseModel):
22     id: int
23     nama: str
24     jenis: str
25     alamat: Optional[str]
26     longitude: float
27     latitude: float
28
29
30 class UserRegister(BaseModel):
31     email: str = Field(..., min_length=5)
32     password: str = Field(..., min_length=6)
33
34
35 class UserResponse(BaseModel):
36     id: int
37     email: str
38
39
40 class TokenResponse(BaseModel):
41     access_token: str
42     token_type: str = "bearer"
```

Gambar Model validasi data pada file models.py

4. Mengimplementasikan Autentikasi JWT

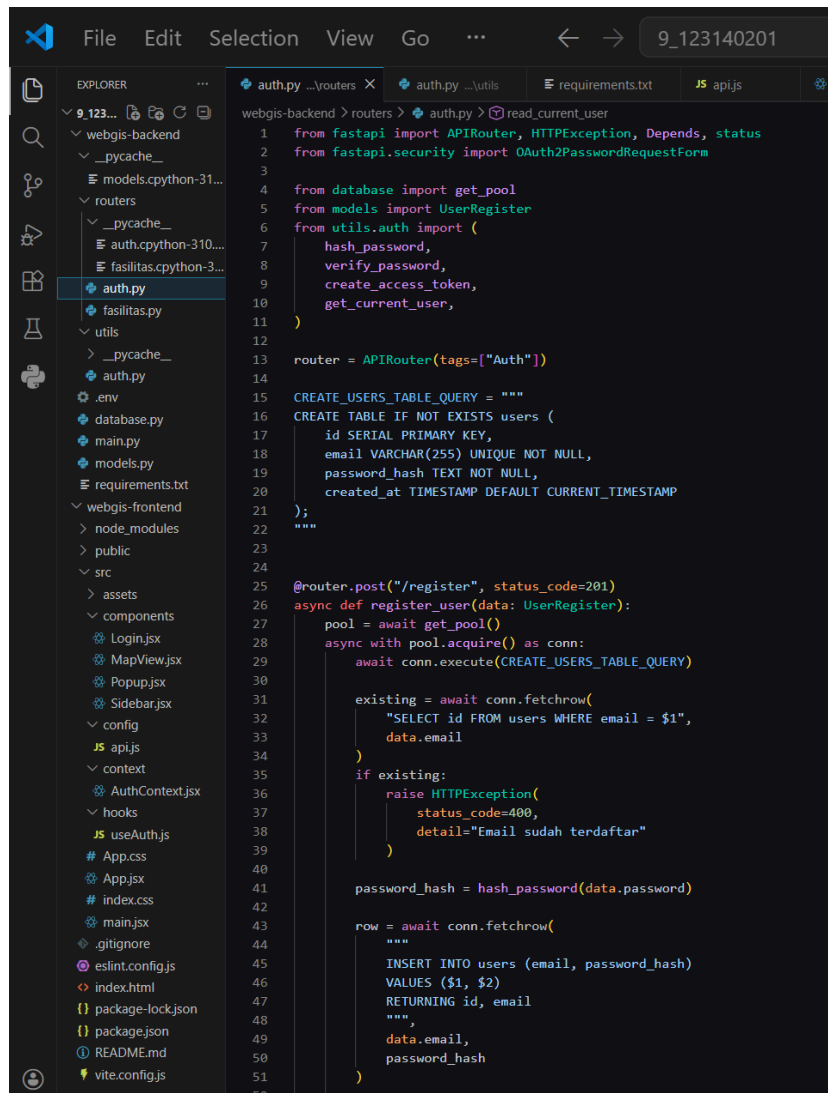
Pada tahap ini ditambahkan autentikasi berbasis JWT. Terdapat tiga komponen utama:

- POST /register untuk membuat akun user baru,
- POST /login untuk menghasilkan access_token,
- GET /me untuk menguji user yang sedang login.

Utility JWT diletakkan pada utils/auth.py, yang berisi:

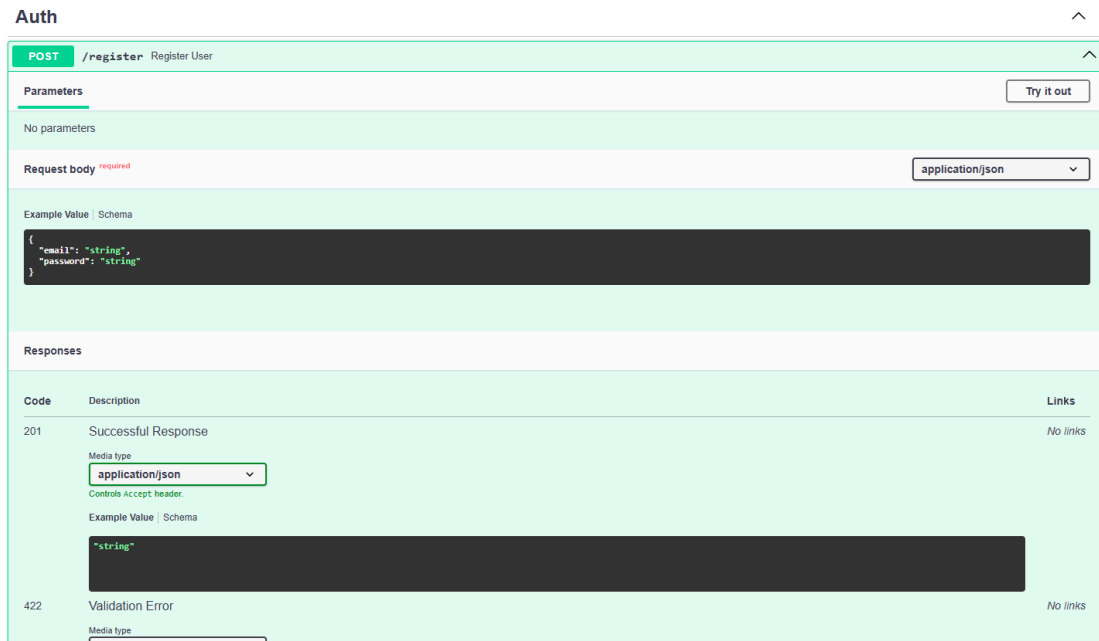
- hashing password,
- verifikasi password,
- pembuatan token,
- verifikasi token melalui dependency.

Pertemuan 9 memang menjelaskan implementasi JWT dengan python-jose, OAuth2PasswordBearer, serta penggunaan token pada header Authorization: Bearer Endpoint login juga dijelaskan menggunakan OAuth2PasswordRequestForm.

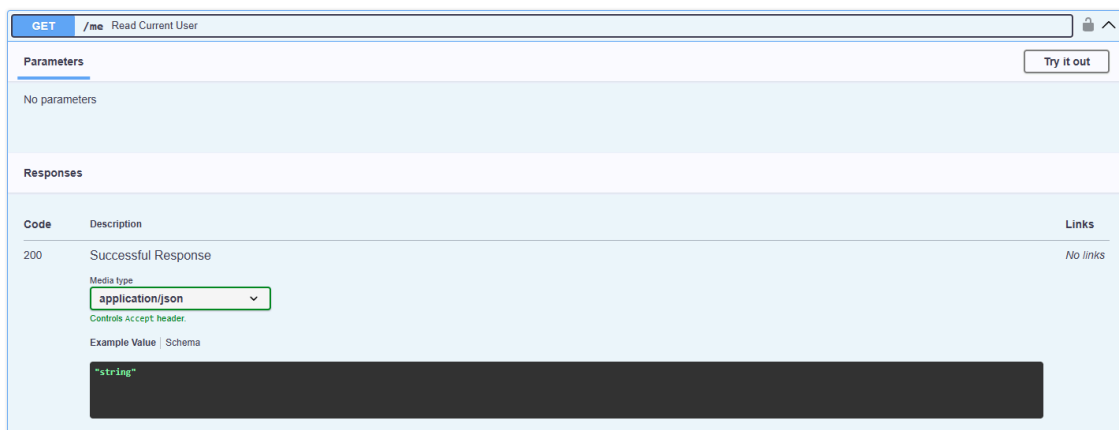


```
1 from fastapi import APIRouter, HTTPException, Depends, status
2 from fastapi.security import OAuth2PasswordRequestForm
3
4 from database import get_pool
5 from models import UserRegister
6 from utils.auth import (
7     hash_password,
8     verify_password,
9     create_access_token,
10    get_current_user,
11)
12
13 router = APIRouter(tags=["Auth"])
14
15 CREATE_USERS_TABLE_QUERY = """
16 CREATE TABLE IF NOT EXISTS users (
17     id SERIAL PRIMARY KEY,
18     email VARCHAR(255) UNIQUE NOT NULL,
19     password_hash TEXT NOT NULL,
20     created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
21 );
22 """
23
24
25 @router.post("/register", status_code=201)
26 async def register_user(data: UserRegister):
27     pool = await get_pool()
28     async with pool.acquire() as conn:
29         await conn.execute(CREATE_USERS_TABLE_QUERY)
30
31     existing = await conn.fetchrow(
32         "SELECT id FROM users WHERE email = $1",
33         data.email
34     )
35     if existing:
36         raise HTTPException(
37             status_code=400,
38             detail="Email sudah terdaftar"
39         )
40
41     password_hash = hash_password(data.password)
42
43     row = await conn.fetchrow(
44         """
45         INSERT INTO users (email, password_hash)
46         VALUES ($1, $2)
47         RETURNING id, email
48         """
49         ,
50         data.email,
51         password_hash
52     )
```

Gambar Implementasi router autentikasi



Gambar register/login



Gambar endpoint /me

5. Menjalankan Backend dan Menguji Endpoint Auth di Swagger

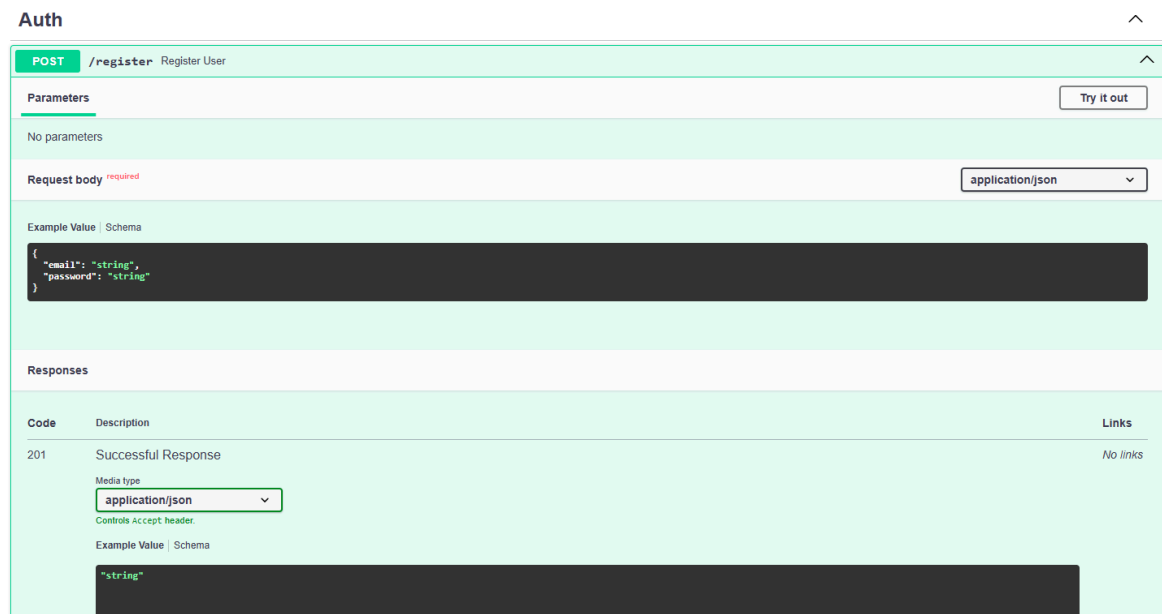
Setelah auth ditambahkan, backend dijalankan melalui terminal menggunakan `uvicorn main:app --reload`. Pengujian dilakukan di Swagger UI melalui endpoint:

- POST /register
- POST /login
- GET /me

Pada tahap register, user baru berhasil dibuat dengan response berisi id dan email. Setelah login berhasil, backend mengembalikan `access_token` dan `token_type: bearer`. Token tersebut kemudian digunakan pada tombol Authorize di Swagger, lalu endpoint /me berhasil mengembalikan email user yang sedang login. Hal ini menunjukkan bahwa mekanisme JWT telah berjalan dengan baik.

```
PS C:\Users\awise\Downloads\9_123140201\webgis-backend> uvicorn main:app --reload
INFO: Will watch for changes in these directories: ['C:\\Users\\awise\\Downloads\\9_123140201\\webgis-backend']
INFO: Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO: Started reloader process [11468] using StatReload
AUTH ROUTER LOADED FROM: C:\\Users\\awise\\Downloads\\9_123140201\\webgis-backend\\routers\\auth.py
INFO: Started server process [5660]
INFO: Waiting for application startup.
Mencoba menyambung ke database...
Berhasil tersambung!
```

Gambar 8. Backend FastAPI berjalan di terminal



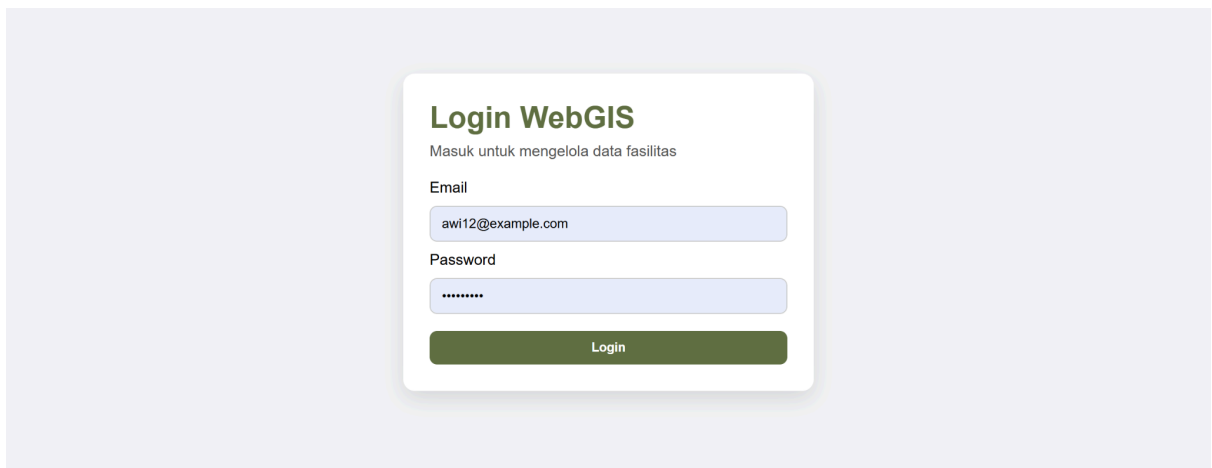
Gambar Proses register user di Swagger UI

6. Menyiapkan Frontend dan Integrasi Login

Frontend yang sudah dibangun pada tugas sebelumnya dikembangkan dengan menambahkan:

- Login.jsx
- AuthContext.jsx
- useAuth.js
- interceptor token pada config/api.js

api.js digunakan untuk menambahkan token otomatis ke setiap request melalui Axios interceptor. AuthContext.jsx digunakan untuk menyimpan status user login, fungsi login, fungsi logout, dan pengecekan sesi login. Login.jsx digunakan sebagai halaman awal yang muncul sebelum user berhasil masuk ke dashboard peta. Materi Pertemuan 9 memang mencontohkan api.js dengan interceptor token dan AuthContext untuk state auth global.



Gambar Halaman login pada frontend

7. Menghubungkan Frontend dengan Peta dan Data GeoJSON

Setelah login berhasil, user diarahkan ke halaman utama yang memuat komponen peta. Komponen `MapView.jsx` mengambil data dari endpoint `GET /api/fasilitas/geojson`, lalu menampilkannya sebagai layer GeoJSON menggunakan `React-Leaflet`. Data marker diberi warna berbeda berdasarkan kategori fasilitas, dan popup menampilkan nama, jenis, serta alamat. Endpoint GeoJSON pada backend memang mengirim data dalam bentuk `FeatureCollection` yang berisi `id`, `nama`, `jenis`, dan `alamat` di dalam `properties`, sehingga sesuai untuk kebutuhan frontend peta.

8. Menambahkan Form Add/Edit di Sidebar

Untuk memenuhi ketentuan tugas, frontend dilengkapi dengan `Sidebar.jsx` yang berisi form tambah dan edit data fasilitas. Form ini memuat field:

- nama
- jenis
- alamat
- longitude
- latitude

Jika user menambah data baru, form akan memanggil endpoint `POST /api/fasilitas/`. Jika user mengedit data yang dipilih dari peta, form akan beralih ke mode edit dan memanggil endpoint `PUT /api/fasilitas/{id}`. Setelah data berhasil disimpan, peta akan direfresh agar perubahan langsung terlihat. Ketentuan tugas memang mewajibkan frontend memiliki form untuk add/edit fasilitas dengan validasi.

Tambah Fasilitas

Isi form untuk menambahkan fasilitas baru.

Nama

Jenis

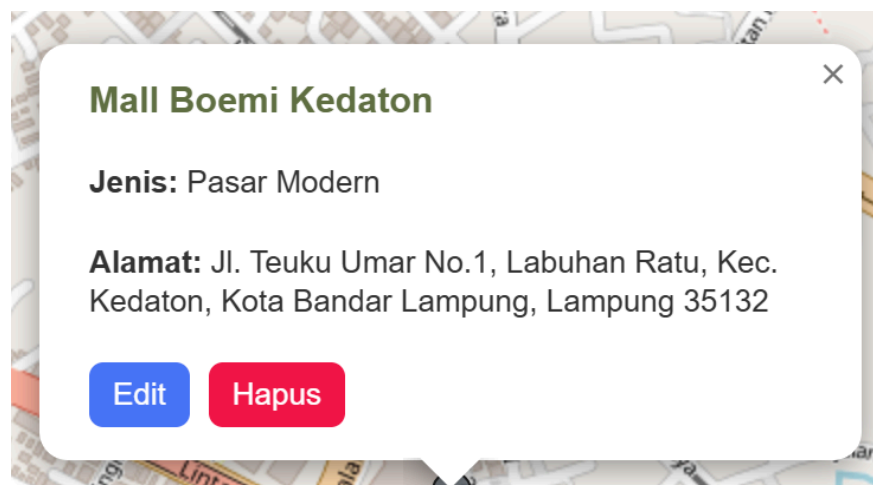
Alamat

Longitude

Latitude

9. Menambahkan Edit dan Delete dari Marker

Marker pada peta dikembangkan agar saat popup dibuka, user bisa menekan tombol Edit atau Hapus. Tombol Edit mengirim data marker ke sidebar sehingga form terisi otomatis. Tombol Hapus memanggil endpoint DELETE `/api/fasilitas/{id}` setelah user memberikan konfirmasi. Dengan cara ini, marker tidak hanya berfungsi sebagai penampil informasi, tetapi juga sebagai pintu masuk untuk manajemen data. Ketentuan tugas secara eksplisit memang meminta marker pada peta dapat diklik untuk edit/delete.

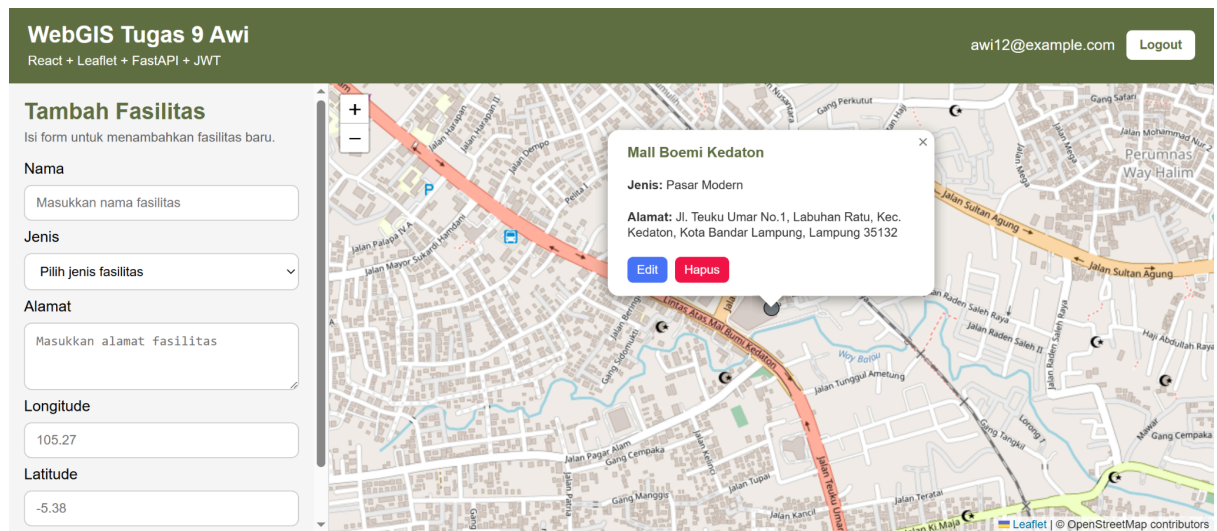


Gambar Tampilan popup marker dengan tombol Edit dan Hapus

10. Hasil Pengujian Fitur CRUD

Fitur CRUD diuji melalui aplikasi frontend dan backend. Hasil pengujian menunjukkan:

- data fasilitas dapat ditambahkan melalui form sidebar,
- data dapat diedit melalui marker dan form edit,
- data dapat dihapus melalui popup marker,
- peta berhasil direfresh setelah perubahan data.



Gambar Final Hasil Tugas 9